

🔑 Objectifs de la feuille

- La console d'administration DJANGO
- Rendre modifiable son application
- Les actions du CRUD
- Personnalisation de sa console d'administration
- DJANGO Toolbar

Introduction au site d'administration de DJANGO

La génération de sites d'administration pour votre équipe ou vos clients pour ajouter, modifier et supprimer du contenu est un travail pénible qui ne requiert pas beaucoup de créativité. C'est pour cette raison que DJANGO automatise entièrement la création des interfaces d'administration pour les modèles. DJANGO a été écrit dans un environnement éditorial, avec une très nette séparation entre les « éditeurs de contenu » et le site « public ». Les gestionnaires du site utilisent le système pour ajouter des nouvelles, des histoires, des événements, des résultats sportifs, etc., et ce contenu est affiché sur le site public. DJANGO résout le problème de création d'une interface uniforme pour les administrateurs du site qui éditent le contenu.

L'interface d'administration n'est pas destinée à être utilisée par les visiteurs du site ; elle est conçue pour les gestionnaires du site.

Nous avons d'abord besoin de créer un utilisateur qui peut se connecter au site d'administration. Lancez la commande suivante : `manage.py createsuperuser`

Saisissez le nom d'utilisateur souhaité et appuyez sur retour. On vous demande alors de saisir l'adresse de courriel souhaitée. L'étape finale est de saisir le mot de passe. On vous demande de le saisir deux fois, la seconde fois étant une confirmation de la première.

Le site d'administration de DJANGO est activé par défaut. Lançons le serveur de développement et explorons-le. À présent, ouvrez un navigateur Web et allez à l'URL « /admin/ » de votre domaine local – par exemple, <http://127.0.0.1:8000/admin/>. Vous devriez voir l'écran de connexion à l'interface d'administration .

Essayez maintenant de vous connecter avec le compte administrateur que vous avez créé à l'étape précédente. Vous devriez voir apparaître la page d'accueil du site d'administration de DJANGO. Vous devriez voir quelques types de contenu éditables : groupes et utilisateurs. Ils sont fournis par `django.contrib.auth`, le système d'authentification livré avec DJANGO.

Rendre l'application modifiable via l'interface d'admin

Mais où est notre application `monApp` ? Elle n'est pas affichée sur la page d'index de l'interface d'administration. Plus qu'une chose à faire : il faut indiquer à l'admin que les objets `Produit` ont une interface d'administration. Pour ceci, ouvrez le fichier `monApp/admin.py` et éditez-le de la manière suivante :

```
from django.contrib import admin
from .models import Produit
```

```
admin.site.register(Produit)
```

Actualisez votre page et c'est magique !

Pour créer une entrée avec l'interface d'administration Cliquons sur **Produits**. On remarque la présence d'un bouton **Ajouter Produit**, ce qui permet d'ajouter une entrée dans la base pour ce modèle. Cliquons dessus. Remplissons les champs que nous avons définis dans notre modèle puis cliquons sur **Enregistrer**. Voilà nous avons créé un produit en quelques clics sans créer aucune interface.

Vous pouvez bien évidemment modifier l'item que vous voulez. Cliquons sur une entrée au hasard et modifiez le code du produit par exemple, puis validez la sauvegarde en restant sur la fiche produit via le bouton **Enregistrer et continuer les modifications**. Ensuite, cliquons sur le bouton **Historique**. On y voit un historique complet sur les modifications de l'entrée, ainsi que le nom de l'utilisateur qui a fait les changements.

À noter ici :

- Le formulaire est généré automatiquement à partir du modèle `Produit`.
- Les différents types de champs du modèle (`DateTimeField`, `CharField`) correspondent au composant graphique d'entrée HTML approprié. Chaque type de champ sait comment s'afficher dans l'interface d'administration de DJANGO.
- Chaque `DateTimeField` reçoit automatiquement des raccourcis Javascript. Les dates obtiennent un raccourci « Aujourd'hui » et un calendrier en popup, et les heures obtiennent un raccourci « Maintenant » et une popup pratique qui liste les heures couramment saisies.

La partie inférieure de la page vous propose une série d'opérations :

- **Enregistrer** – Enregistre les modifications et retourne à la page liste pour modification de ce type d'objet.
- **Enregistrer et continuer les modifications** – Enregistre les modifications et recharge la page d'administration de cet objet.
- **Enregistrer et ajouter un nouveau** – Enregistre les modifications et charge un nouveau formulaire vierge pour ce type d'objet.
- **Supprimer** – Affiche une page de confirmation de la suppression.

D'ailleurs, supprimez le produit que vous avez potentiellement créé et modifié.

Qu'est-ce que le CRUD ?

L'interface d'administration DJANGO est CRUD (Create Read Update et Delete), c'est à dire qu'elle est capable de faire les opérations de bases :

/admin/monApp/	=> liste les modèles de l'application
/admin/monApp/produits/	=> liste les objets du modèle
/admin/monApp/produit/add	=> crée un nouveau objet
/admin/monApp/produit/<id>/	=> consulter un objet en particulier
/admin/monApp/produit/<id>/delete	=> supprime un objet en particulier
/admin/monApp/produit/<id>/change	=> modifie un objet en particulier

C'est cool, mais quel est le but du site d'administration ?

Le site d'administration est un endroit où les différents modèles d'un projet DJANGO peuvent être gérés par, eh bien, des administrateurs ! Mais qui entendons-nous par là ? Pour commencer, les administrateurs ce sera vous et les autres développeurs du projet. Mais vous pouvez éventuellement confier le projet à un client : peut-être le propriétaire d'un magasin ou l'auteur d'un blog, des personnes qui ne sont pas des programmeurs et qui ne peuvent donc pas utiliser le shell DJANGO pour créer de nouveaux objets. Les non-programmeurs seraient perdus sans une interface permettant d'effectuer des opérations CRUD. Et vous devrez passer un temps considérable à construire et à tester cette interface pour eux. C'est pourquoi vous devriez vous réjouir de cette fonctionnalité, car DJANGO l'a conçue pour vous !

Cela signifie-t-il que vous n'aurez jamais à construire vos propres formulaires ? Pas tout à fait ! Si le site d'administration est très utile, il ne faut pas oublier que son public principal est constitué d'administrateurs et non d'utilisateurs finaux. Il s'agit d'une interface « back-end ». Il offre des fonctionnalités bien supérieures à celles auxquelles les utilisateurs finaux devraient avoir accès. Il est aussi très simple d'aspect.

Vos utilisateurs finaux s'attendent à utiliser des formulaires dont le style est identique à celui du reste de votre site. Vous pouvez également personnaliser la position d'un formulaire dans une page, exclure certains champs d'un formulaire et effectuer d'autres personnalisations de l'interface utilisateur. Pour ces raisons, vous devrez encore apprendre à intégrer des formulaires dans le front-end de votre application web dans les chapitres suivants.

Personnaliser son interface d'administration

Il existe une batterie d'options qui vous permettront de mettre en avant les informations qui vous intéressent. Si nous voulons la liste des `Produit` via deux colonnes, `intituleProd` et `prixUnitaireProd`, éditez `monApp/admin.py` de la manière suivante avec le champ `list_display` :

```
from django.contrib import admin
from .models import Produit

class ProduitAdmin(admin.ModelAdmin):
    list_display = ('intituleProd', 'prixUnitaireProd')

admin.site.register(Produit, ProduitAdmin)
```

Les options de `ModelAdmin` : <https://docs.djangoproject.com/fr/5.1/ref/contrib/admin/#modeladmin-options>

Prenez le modèle disponible sur Célène (c'est normalement le même que le votre si vous avez tout bien fait jusque ici). Dans le cas contraire, si changement il y a, n'oubliez pas les étapes de migrations. Mais nous devrions avoir tous le même modèle dorénavant à partir de maintenant. Faites en sorte que toutes les entités du modèle s'affichent dans la console d'administration en rajoutant dans le fichier `monApp/admin.py` les lignes suivantes :

```
admin.site.register(Categorie)
admin.site.register(Statut)
admin.site.register(Rayon)
```

Nous voulons pouvoir créer des produits directement sur la fiche catégorie. Le fichier `monApp/admin.py` est donc modifié en rajoutant :

```
class ProduitInline(admin.TabularInline):
    model = Produit
    extra = 1 # nombre de lignes vides par défaut
```

Puis en modifiant :

```
class CategorieAdmin(admin.ModelAdmin):
    model = Categorie
    inlines = [ProduitInline]
```

`ProduitInline` hérite de `admin.TabularInline` (ou `admin.StackedInline` si tu préfères une présentation en blocs). On l'attache à `CategorieAdmin` pour qu'en éditant une catégorie, on voie tous ses produits. `ProduitAdmin` reste un admin normal pour gérer les produits séparément. Voilà notre fiche catégorie avec possibilité de créer des produits très simplement.

```
admin.site.register(Categorie, CategorieAdmin)
```

La première chose que l'on remarque c'est la facilité du code pour créer une interface complète avec ajout / modification / suppression / contrôles de données. Il nous aura fallu peu de lignes de code, sans compter les modèles.

Sur la page des produits, on peut rendre éditable maintenant les colonnes `intituleProd`, `prixUnitaireProd`, et `dateFabProd` directement dans le listing à l'aide du champ `list_editable` :

```
class ProduitAdmin(admin.ModelAdmin):
    model = Produit
    list_display = ["refProd", "intituleProd", "prixUnitaireProd", "dateFabProd", "categorie", "statut"]
    list_editable = ["intituleProd", "prixUnitaireProd", "dateFabProd"]
```

Lors de l'ajout d'un produit (cliquez sur le bouton), si la liste déroulante pour le statut ne vous plaît pas (nous avons peu de valeurs), vous pouvez la remplacer par des boutons radio comme ceci:

```
class ProduitAdmin(admin.ModelAdmin):
    model = Produit
    list_display = ["refProd", "intituleProd", "prixUnitaireProd", "dateFabProd", "categorie", "statut"]
    list_editable = ["intituleProd", "prixUnitaireProd", "dateFabProd"]
    radio_fields = {"statut": admin.VERTICAL}
```

Sur la page des produits par exemple, il est possible d'ajouter une searchbox qui effectue des recherches sur plusieurs champs:

```
class ProduitAdmin(admin.ModelAdmin):
    model = Produit
    list_display = ["refProd", "intituleProd", "prixUnitaireProd", "dateFabProd", "categorie", "statut"]
    list_editable = ["intituleProd", "prixUnitaireProd", "dateFabProd"]
    radio_fields = {"statut": admin.VERTICAL}
    search_fields = ('intituleProd', 'dateFabProd')
```

Notre exemple de recherche précédent ressemble assez à du bricolage. Il existe une autre option qui vous permet de filtrer les données plus proprement (avec une jolie interface sur la droite):

```
class ProduitAdmin(admin.ModelAdmin):
    model = Produit
    list_display = ["refProd", "intituleProd", "prixUnitaireProd", "dateFabProd", "categorie", "statut"]
    list_editable = ["intituleProd", "prixUnitaireProd", "dateFabProd"]
    radio_fields = {"statut": admin.VERTICAL}
    search_fields = ('intituleProd', 'dateFabProd')
    list_filter = ('statut', 'dateFabProd')
```

Vous pouvez affiner votre filtre en utilisant la classe `admin.SimpleListFilter`:

```
class ProduitFilter(admin.SimpleListFilter):
```

```
    title = 'filtre produit'
    parameter_name = 'custom_statut'

    def lookups(self, request, model_admin) :
        return (
            ('OnLine', 'En ligne'),
            ('OffLine', 'Hors ligne'),
        )

    def queryset(self, request, queryset):
        if self.value() == 'OnLine':
            return queryset.filter(statut=1)
        if self.value() == 'OffLine':
            return queryset.filter(statut=0)
```

```
class ProduitAdmin(admin.ModelAdmin):
    model = Produit
    list_display = ["refProd", "intituleProd", "prixUnitaireProd", "dateFabProd", "categorie", "statut"]
    list_editable = ["intituleProd", "prixUnitaireProd"]
    radio_fields = {"statut": admin.VERTICAL}
    search_fields = ('intituleProd', 'dateFabProd')
    list_filter = (ProduitFilter,)
```

Vous pouvez hiérarchiser par date :

```
class ProduitAdmin(admin.ModelAdmin):
    model = Produit
    list_display = ["refProd", "intituleProd", "prixUnitaireProd", "dateFabProd", "categorie", "statut"]
    list_editable = ["intituleProd", "prixUnitaireProd"]
    radio_fields = {"statut": admin.VERTICAL}
    search_fields = ('intituleProd', 'dateFabProd')
    list_filter = (ProduitFilter,)
    date_hierarchy = 'dateFabProd'
```

Vous remarquerez la présence d'un lien au dessus des actions qui vous permet de naviguer / filtrer par des intervalles de date.

Vous pouvez trier par défaut en utilisant le signe - pour indiquer un ordre décroissant :

```
class ProduitAdmin(admin.ModelAdmin):  
    model = Produit  
    list_display = ["refProd", "intituleProd", "prixUnitaireProd", "dateFabProd", "categorie", "statut"]  
    list_editable = ["intituleProd", "prixUnitaireProd"]  
    radio_fields = {"statut": admin.VERTICAL}  
    search_fields = ('intituleProd', 'dateFabProd')  
    list_filter = (ProduitFilter,)  
    date_hierarchy = 'dateFabProd'  
    ordering = ('-dateFabProd',)
```

Vous pouvez ajouter une action dans la liste des actions très simplement avec le module admin. Par exemple nous voulons changer le statut des produits sélectionnés en **Offline** (idStatus=0) et **Online** (idStatus=1) :

```
def set_Produit_online(modeladmin, request, queryset):  
    queryset.update(statut=1)  
set_Produit_online.short_description = "Mettre en ligne"  
  
def set_Produit_offline(modeladmin, request, queryset):  
    queryset.update(statut=0)  
set_Produit_offline.short_description = "Mettre hors ligne"
```

```
class ProduitAdmin(admin.ModelAdmin):  
    model = Produit  
    list_display = ["refProd", "intituleProd", "prixUnitaireProd", "dateFabProd", "categorie", "statut"]  
    list_editable = ["intituleProd", "prixUnitaireProd"]  
    radio_fields = {"statut": admin.VERTICAL}  
    search_fields = ('intituleProd', 'dateFabProd')  
    list_filter = (ProduitFilter,)  
    date_hierarchy = 'dateFabProd'  
    ordering = ('-dateFabProd',)  
    actions = [set_Produit_online, set_Produit_offline]
```

Je sélectionne l'action puis je clique sur envoyer. Pour les produits sélectionnés, le statut aura

- la valeur 1 comme il est défini dans la fonction **set_Produit_online** que j'ai créée.
- la valeur 0 comme il est défini dans la fonction **set_Produit_offline** que j'ai créée.

Vous pouvez créer une colonne personnalisée et y mettre les informations que vous voulez :

```
class ProduitAdmin(admin.ModelAdmin):
    model = Produit
    list_display = ["refProd", "intituleProd", "prixUnitaireProd", "prixTTCProd", "dateFabProd", "categorie", "statut"]
    list_editable = ["intituleProd", "prixUnitaireProd"]
    radio_fields = {"statut": admin.VERTICAL}
    search_fields = ('intituleProd', 'dateFabProd')
    list_filter = (ProduitFilter,)
    date_hierarchy = 'dateFabProd'
    ordering = ('-dateFabProd',)
    actions = [set_Produit_online, set_Produit_offline]

    def prixTTCProd(self, instance):
        return (instance.prixUnitaireProd * Decimal('1.20')).quantize(Decimal('0.01'), rounding=ROUND_HALF_UP)

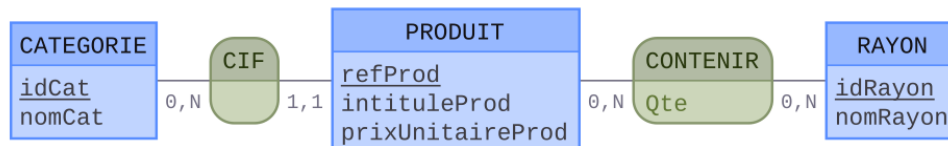
    prixTTCProd.short_description = "Prix TTC"
```

A noter que la colonne n'est pas triable. Pour cela, il faut rajouter la ligne suivante dans la classe **ProduitAdmin** : `prixTTCProd.admin_order_field = "prixUnitaireProd"`

Voilà les choses essentielles que nous voulions voir dans cette interface d'administration. Il nous manque cependant quelques chose...

Evolution du modèle

Si vous vous rappelez le MCD vu précédemment dans le TP 1, nos rayons sont censés contenir des produits, et des produits peuvent se retrouver dans plusieurs rayons, tout ceci selon une certaine quantité.



Ce type d'association se traduit logiquement dans votre modèle par une table dont :

- le nom est le nom de l'association,
- la clé primaire de cette table est la composition unique des clés étrangères des entités qu'elle relie.
- la donnée portée par l'association est un champ de la table

Ci dessous la traduction et la modification à apporter dans votre fichier `monApp/models.py` :

```
class Contenir(models.Model):
    produit = models.ForeignKey(Produit, on_delete=models.CASCADE, related_name="contenir_produit")
    rayon = models.ForeignKey(Rayon, on_delete=models.CASCADE, related_name="contenir_rayon")
    Qte = models.PositiveIntegerField()

    class Meta:
        constraints = [models.UniqueConstraint(fields=['produit', 'rayon'], name='unique_produit_rayon')]

    def __str__(self):
        return f"{self.produit} dans {self.rayon} (Qte: {self.Qte})"
```

Vous pouvez également indiquer dans **Produit** la relation Many-To-Many avec **Rayon** via **Contenir**. Et comme la relation **Contenir** possède un attribut supplémentaire qu'est la quantité, il faut utiliser l'option **through**. Cette écriture dans **Produit** supprimerait la **class Meta** dans **Contenir** :

```
class Produit(models.Model):
    refProd = models.AutoField(primary_key=True)
    intituleProd = models.CharField(max_length=200)
    prixUnitaireProd = models.DecimalField(max_digits=10, decimal_places=2)
    dateFabProd = models.DateField(default=date.today)

    # Relation CIF : chaque produit appartient à 1 catégorie (0,N côté catégorie → 1,1 côté produit)
    categorie = models.ForeignKey(Categorie, on_delete=models.CASCADE, related_name="produit_categorie", null=True, blank=True)
    status = models.ForeignKey(Statut, on_delete=models.CASCADE, related_name="produit_statut", null=True, blank=True)

    # Relation ManyToMany avec Rayon, via Contenir
    rayons = models.ManyToManyField("Rayon", through="Contenir", related_name="produits")

    def __str__(self):
        return self.intituleProd
```

Enfin pour les indécis, vous pouvez indiquer dans **Rayon** la relation Many-To-Many avec **Produit** via **Contenir**... (cette écriture dans **Rayon** supprimerait aussi la **class Meta** dans **Contenir**)

```
class Rayon(models.Model):
    idRayon = models.AutoField(primary_key=True)
    nomRayon = models.CharField(max_length=100)

    # Relation ManyToMany avec Produit, via Contenir
    produits = models.ManyToManyField("Produit", through="Contenir", related_name="rayons")

    def __str__(self):
        return self.nomRayon
```

Personnellement, je préfère la toute première écriture qui me semble plus clair et qui demeure la traduction du MCD en MLD.

C'est à vous de jouer !

- Modifiez votre modèle, et faites votre migrations
- Maintenez votre taux de couverture de test (autrement dit, rajoutez des tests sur **Contenir**)
- Faites apparaître **Contenir** dans l'interface d'administration Django
- Associez un produit et un rayon adéquat et choisissez une quantité raisonnable à l'aide de l'interface d'administration de Django. Répétez l'opération pour que les tous rayons soient un peu près tous bien équipés en produits.

Installation de DJANGO DEBUG TOOLBAR

Commande : `pip install django-debug-toolbar`

Dans le fichier `monProj/settings.py`:

```
INSTALLED_APPS = [  
    ...,  
    'debug_toolbar',  
]  
  
MIDDLEWARE = [  
    # ...  
    "debug_toolbar.middleware.DebugToolbarMiddleware",  
    # ...  
]  
  
INTERNAL_IPS = [  
    # ...  
    "127.0.0.1",  
    # ...  
]
```

Dans le fichier `monProj/urls.py` du projet:

```
urlpatterns = [  
    path("monApp/", include("monApp.urls")),  
    path('admin/', admin.site.urls),  
    path("__debug__/", include("debug_toolbar.urls")),  
]
```

Faites une migration et lancer votre application.

DJANGO TOOLBAR est maintenant visible.

Nous allons pouvoir maintenant parler optimisation.

Rendons-nous sur l'url <http://127.0.0.1:8000/admin/monApp/produit/3/change/>

Dans la DJANGO TOOLBAR, il faut 6 requêtes SQL pour afficher la page (2.69 ms)

Rendons-nous sur l'url <http://127.0.0.1:8000/admin/monApp/produit/2/change/>

Dans la DJANGO TOOLBAR, il faut 5 requêtes SQL pour afficher la page (2.34 ms)

Rendons-nous sur l'url <http://127.0.0.1:8000/admin/monApp/produit/>

Dans la DJANGO TOOLBAR, il faut 15 requêtes SQL pour afficher la page (3.77 ms)

Dans certain cas, il peut être intéressant de ne pas afficher un select mais plutôt un input simple . Dans cet input il faudra indiquer la clé de l'item que l'on désire associer au lieu de le sélectionner manuellement dans une liste. Le cas le plus évident sera les cas où les items seraient si nombreux que le chargement de la page serait trop lent. Cette option permet donc de répondre à ce genre de besoin. Nous y reviendrons plus tard quand il sera temps de faire de l'optimisation.